William Mallard, PhD

Palo Alto, CA · wjm@llard.net · wjmallard.net · github.com/wjmallard · linkedin.com/in/wjmallard

I bring rigorous biological training and deep computational expertise. I design and build scalable data analysis pipelines that are tightly intertwined with the underlying biology.

Skills

Languages	Python, C, C++, Bash, CUDA, Assembly, Java, MATLAB, R, Unix text processing
Libraries	NumPy, SciPy, Pandas, Matplotlib, Seaborn, PyTorch, Scikit-learn, Scikit-image
Bio Tools	AlphaFold, PyMOL, nmrglue, bedtools, HMMER, Napari, Fiji, BioIO
Systems	Linux/Unix, HPC (SLURM, LSF), PostgreSQL, GPUs, FPGAs, Git
Domains	Computational Biology, Multi-omics, Quantitative Image Analysis, Structural Biology
Methods	Machine Learning, Computer Vision, Agentic RAG (MCP), Real-time Signal Processing

Experience

Harvard University — *Cambridge, MA*
Doctoral Researcher, Molecular and Cellular Biology — *8/2016 – 12/2025*

I discovered a novel regulatory mechanism in bacterial cell division (PLoS One, 2025; co-corresponding author).

- **Novel IDR contact site discovery:** Developed a computational framework for predicting interactions between folded domains and intrinsically disordered regions (IDRs) using ensembles of AlphaFold-Multimer models to statistically identify contact residues. Validated my *in silico* predictions experimentally via ITC and quantitative 2D NMR.
- **High-throughput cell morphometric measurements:** Designed and performed live-cell (wide-field, TIRF, confocal) imaging experiments — from microscopy to quantitative image analysis pipelines. Developed computer vision and computational geometry algorithms to quantify cell shape and subcellular dynamics; hand-built and optimized core algorithms in vectorized NumPy; profiled to eliminate bottlenecks; parallelized processing on a SLURM cluster.
- **Proximity-labeling candidate stratification:** Devised a strategy for ranking candidate lipid-droplet-surface proteins from time-resolved proximity-labeling mass-spec, using enrichment kinetics to distinguish recruitment to the droplet surface from static background; 17 of the 20 top-ranked candidates were experimentally validated *in vivo*.

Broad Institute of MIT and Harvard — *Cambridge, MA*
Computational Biologist, Rinn Lab — *2/2013 – 7/2016*

I was a core computational analyst on 11 publications.

- **Functional genomics:** Developed custom integrative multi-omics analyses (RNA-seq, ChIP-seq, RIP-seq, ATAC-seq) to extract molecular mechanisms from perturbations and time courses of stem cell and developmental models; iterated with bench collaborators on experimental design and interpretation.
- **Quantitative biology:** Designed statistical metrics to measure cell-biological features, e.g., mRNA splicing efficiency, export efficiency, and tissue specificity.
- **Computational infrastructure:** Built the lab's workhorse lncRNA discovery pipeline.

Broad Institute of MIT and Harvard — *Cambridge, MA*
Software Engineer, The Cancer Genome Atlas — *4/2012 – 2/2013*

Worked on the GDAC Firehose, a production genomics platform processing ∼14,000 samples across 25 cancer types.

- Identified and eliminated bottlenecks, refactoring or rewriting omics analyses to run at scale.
- Traced one production failure to the Python standard library, and contributed upstream fixes to CPython 3.4 — enabling ZIP64 archives in shutil (#17189) and making them the default in zipfile (#17201).

Earlier Engineering Experience

Space Sciences Laboratory — *Digital Design Engineer*, Townes Lab — *Berkeley, CA · 9/2009 – 8/2011*
Center for Astronomy Signal Processing & Electronics Research — *Digital Design Engineer* — *Berkeley, CA · 4/2008 – 8/2011*
MusicianLink — *Software Engineer* — *San Jose, CA · 6/2007 – 3/2008*
UC Berkeley Clustered Computing — *Unix Systems Administrator* — *Berkeley, CA · 1/2006 – 8/2006*
Massachusetts Institute of Technology — *Research Assistant*, Chuang Lab — *Cambridge, MA · 6/2005 – 8/2005*
University of California, Berkeley — *Research Assistant*, Holzapfel Lab — *Berkeley, CA · 6/2004 – 5/2005*
Berkeley Student Cooperative — *Unix Network Administrator* — *Berkeley, CA · 8/2003 – 12/2006*

Education

Harvard University — PhD, Biochemistry — *Cambridge, MA · 5/2025*
University of California, Berkeley — BS, Engineering Physics; *minors in CS and Mathematics* — *Berkeley, CA · 5/2007*

Recent Projects

bacterial phospho-site predictor (2026) — Trained an MLP on frozen ESM-2 embeddings to predict phospho-sites in bacteria, sweeping hyperparameters to find a minimal architecture. Discovered that a published ten-feature ensemble's benchmark was inflated by train-on-test contamination; showed that my simpler model matched its performance when both were run on a properly split version of their dataset.

bio knowledge graph (2026) — Built a knowledge graph over seven decades of bacterial cell-division literature (∼8k papers), ingesting PubMed abstracts, PMC full text, and scanned pre-digital PDFs via local OCR (MinerU). Tags biomolecules with NER and links them through published vocabularies and ontologies. Co-mention edges point back to their source sentences. Disambiguates authors and institutions. A citation-lineage layer recovers a gene's literature from before its name stabilized.

discourse-analysis (2026) — Built a search engine over a Twitter corpus (∼500k posts) unifying full-text, fuzzy, and semantic search. Runs image OCR (Tesseract), machine translation (NLLB-200), and embedding (Qwen) on-device at zero cloud cost. Retrieves posts and reconstructs their context (threads, replies, quotes). Exposes an agentic tool interface (MCP) tailored to how an LLM explores a corpus.

gpu-xengine (2026) — Ported a matrix multiplication engine for real-time signal cross-correlation from FPGA to CUDA. Optimized memory access patterns (coalescing, pinning) for a 1.5× throughput gain.

Earlier Projects

ISI digital backend (2011) — Real-time FX correlator for the Infrared Spatial Interferometer at Mount Wilson Observatory.

- Designed a 138 Gbps signal processing system across 3 synchronized Xilinx Virtex-5 FPGAs, channelizing 2.88 GHz of bandwidth into 64 spectral channels with real-time cross-correlation.
- Achieved a 1.8× clock speedup (200 → 360 MHz) over all prior CASPER designs by mapping core FFT arithmetic onto DSP48Es and developing a BRAM/DSP floorplanning methodology.
- Contributed 50+ optimized DSP commits upstream to CASPER's open-source FPGA library; documented the floorplanning methodology in CASPER Memo #42.
- Built an end-to-end data acquisition pipeline, from FPGA fabric to PowerPC firmware to Python control software and networked storage.
- Instrument resolved individual molecular signatures in Betelgeuse's atmosphere; results published in Wishnow et al. 2010 (SPIE); code and documentation available on GitHub.

CASPER FPGA toolkit (2009) — Contributed to CASPER, an open-source FPGA toolkit for radio astronomy signal processing.

- Refactored and extended Simulink blocks and mask scripts in the CASPER FPGA library.
- Developed a 10 Gbps UDP receiver with a synchronized ring buffer; the core architecture was adopted into PySPEAD, the data transport layer for the Square Kilometre Array.
- Built enclosure and RF signal routing for a spectrometer deployed at Arecibo Observatory.

Quantum ion-trap pulse sequencer (2005) — Built an instrument to programmatically shuttle charged microspheres around a planar Paul trap (Phys. Rev. A, 2006).

- Designed a compact instruction set for precisely timed voltage sequences; wrote a "pulse script" interpreter in ∼1,000 lines of Scenix microcontroller assembly, and a Python client to drive it.
- Built the hardware: salvaged a NIM enclosure, mounted and wired the microcontroller and buffer electronics, designed a front panel in CAD, and cut it from aluminum stock on a waterjet.
- First planar-trap demonstration of the three movement primitives required for a scalable ion-trap quantum computer.

Helium dilution fridge control system (2004) — Wrote a C++ control system for a helium dilution fridge cooling a millimeter-wave telescope detector to 250 mK.

- Rewrote the original LabVIEW system from scratch in C++ for robustness and remote deployment.
- Interfaced with ADC and GPIO boards via Comedi device drivers on Linux.
- The system sequenced the multi-stage cooldown, then held temperature through closed-loop feedback.